

DEVOPS PRACTICAL REFERENCE SERIES

PART 02

Git & GitHub

From object-model internals to production branch protection: the complete Git and GitHub foundation for a DevOps engineer -- commands, recovery drills, workflows, and security.

[Git](#)[GitHub](#)[Branching Strategies](#)[Code Review](#)[Security](#)[Troubleshooting](#)

Audience: DevOps engineers with ~2 years of experience

Format: Deep-dive practical reference — concepts, architecture, configuration, commands, troubleshooting, security, automation, and hands-on labs.

How to use: Read sequentially. Each part is self-contained but builds on earlier parts in the series.

Table of Contents

1. Introduction	5
1.1 What This Part Covers	5
1.2 Why Git, and Why Understanding Internals Matters	5
2. Git Architecture & Internals	6
2.1 What It Is and Why It Exists	6
2.2 The Four Object Types	6
2.3 The .git Directory	7
2.4 Refs, HEAD, and the Working Tree	9
3. Git Fundamentals	10
3.1 Setup and Configuration	10
3.2 Creating and Cloning Repositories	10
3.3 The Staging Area: add, status, diff	11
3.4 Committing	11
3.5 Viewing History	12
4. Branching & Merging	13
4.1 What Branching Is and Why It's Cheap	13
4.2 Merging: Fast-Forward vs Three-Way	13
4.3 Merge Conflicts	14
4.4 Merge Strategies on GitHub	15
5. Rebasing	16
5.1 What It Is and Why It Exists	16
5.2 Interactive Rebase: Cleaning Up History	16
5.3 Rebase Conflicts	17
5.4 Rebase vs Merge: When to Use Which	17
5.5 The Golden Rule of Rebasing	18
6. Undoing Changes	19
6.1 The Three Undo Commands and What Each One Touches	19
6.2 Amending and Splitting Commits	20
6.3 Recovering Lost Work: The Reflog	20
6.4 Cleaning Untracked Files	21
7. Stashing, Cherry-Picking, and Bisecting	22
7.1 git stash: Temporary Shelving of Changes	22
7.2 git cherry-pick: Applying Specific Commits	22
7.3 git bisect: Binary-Searching for a Regression	23
8. Remote Repositories & Collaboration	24
8.1 Remotes	24

8.2 fetch vs pull vs push	24
8.3 Tracking Branches and Force-Pushing Safely	25
8.4 Forking Workflow	25
9. Git Branching Strategies	26
9.1 Why This Matters	26
9.2 Trunk-Based Development	26
9.3 GitHub Flow	26
9.4 Git Flow	27
9.5 Choosing a Strategy	27
10. Tags & Releases	28
10.1 Lightweight vs Annotated Tags	28
10.2 Semantic Versioning	28
11. .gitignore, .gitattributes, and Git LFS	29
11.1 .gitignore	29
11.2 .gitattributes	29
11.3 Git LFS (Large File Storage)	30
12. Git Hooks & Automation	31
12.1 What Hooks Are and Why They Exist	31
12.2 Common Hooks	31
12.3 Sharing Hooks Across a Team: pre-commit Framework	32
12.4 Server-Side Hooks and GitHub Equivalents	33
13. Submodules and Monorepo vs Polyrepo	34
13.1 Git Submodules	34
13.2 Monorepo vs Polyrepo	35
14. GitHub Platform Deep Dive	36
14.1 What GitHub Adds on Top of Git	36
14.2 Authentication: SSH vs HTTPS vs Tokens	36
14.3 Pull Requests and Code Review	37
14.4 Branch Protection Rules	37
14.5 CODEOWNERS	38
14.6 Issues, Labels, Milestones, and Projects	38
14.7 GitHub CLI (gh)	39
15. Git & GitHub Security	40
15.1 What It Is and Why It Exists	40
15.2 Commit Signing (GPG / SSH)	40
15.3 Secret Scanning and Prevention	41
15.4 Deploy Keys vs Personal Access Tokens vs GitHub Apps	41
15.5 Repository-Level Security Settings Checklist	42
16. Troubleshooting Git	43
16.1 A Systematic Approach	43

16.2 Scenario: “I accidentally hard-reset and lost commits”	43
16.3 Scenario: Merge conflict I can’t figure out	43
16.4 Scenario: A secret was committed and needs to be purged from history	44
16.5 Scenario: Accidentally committed to the wrong branch	44
16.6 Scenario: Detached HEAD with commits you want to keep	44
16.7 Scenario: Diverged branches (“your branch and ‘origin/main’ have diverged”)	45
16.8 Diagnostic Command Reference	45
17. Best Practices and Anti-Patterns	46
17.1 Commit Hygiene	46
17.2 Code Review Culture	46
17.3 Repository Hygiene	47
18. Hands-On Labs	48
Lab 1 — Object Model Exploration	48
Lab 2 — Branching, Merging, and Conflict Resolution	48
Lab 3 — Recovery Drills	48
Lab 4 — Interactive Rebase Cleanup	49
Lab 5 — GitHub Collaboration Workflow	49
Lab 6 — Git Hooks and pre-commit	49
Lab 7 — Bisect a Real Regression	49
19. Command & Reference Cheat Sheet	50
19.1 Everyday Commands	50
19.2 Undo & Recovery	50
19.3 Rebase & History	50
19.4 GitHub CLI	51
20. Production Readiness Checklist	52

1. Introduction

1.1 What This Part Covers

Part 02 covers Git — the version control system underlying virtually every software delivery pipeline — and GitHub, the platform most teams use to host, review, and collaborate on Git repositories. This part goes past “add, commit, push”: it covers what Git actually stores on disk, why rebasing and merging produce different histories, how to recover from the mistakes every engineer eventually makes (force-pushed the wrong branch, committed a secret, lost commits after a hard reset), and how GitHub’s collaboration model (PRs, branch protection, CODEOWNERS) is used in real engineering organizations.

HOW TO USE THIS PART

If you already use Git daily, skim Sections 2-4 and focus on Section 5 (rebasing), Section 16 (troubleshooting/recovery), and Section 14 (GitHub platform features) — these are where 2-3 year engineers most often have gaps, and where interviewers probe deepest, because they separate “I use Git” from “I understand Git.”

1.2 Why Git, and Why Understanding Internals Matters

Git is a **distributed version control system**: every clone is a full copy of the repository’s history, not a thin checkout pointing at a central server. This single design decision is why Git supports offline work, why branching is cheap (a branch is just a 41-byte pointer, not a directory copy), and why recovery from mistakes is usually possible — the data you “lost” is very often still sitting in Git’s object database, just unreferenced.

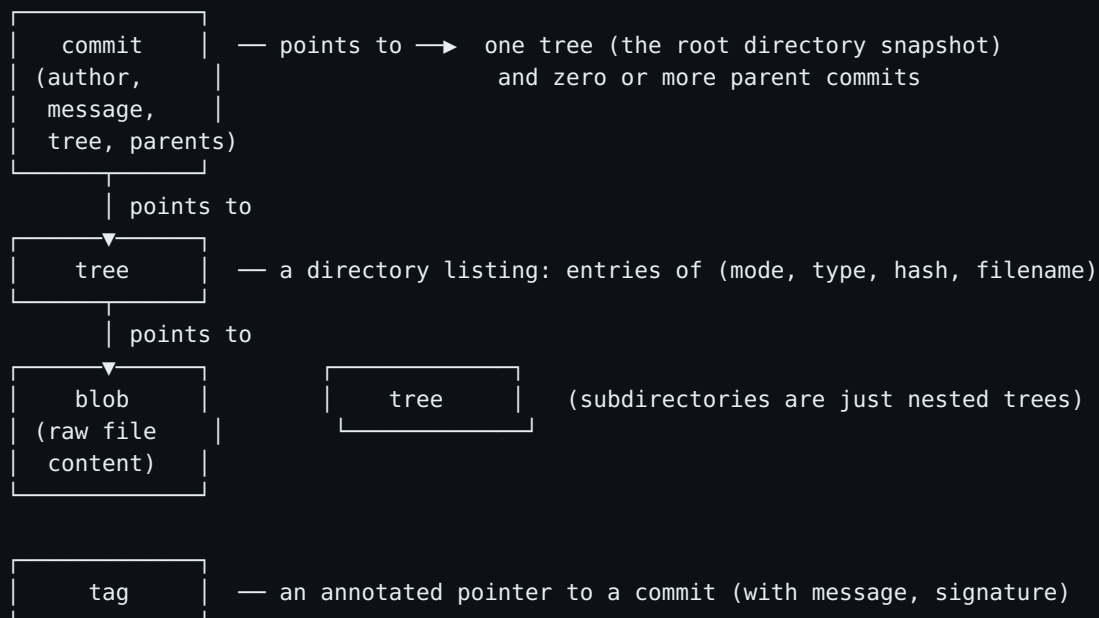
Most engineers learn Git as a sequence of memorized commands. That approach breaks down exactly when it matters most — mid-incident, with a broken history, under time pressure. Understanding what Git actually stores (Section 2) turns every confusing situation (detached HEAD, “diverged branches,” a bad rebase) into something you can reason about directly instead of guessing at magic commands.

2. Git Architecture & Internals

2.1 What It Is and Why It Exists

Git is fundamentally a **content-addressable filesystem with a version history layered on top**. Everything Git tracks — file contents, directory structures, commits — is stored as an **object**, identified by the SHA-1 (or SHA-256 in newer repos) hash of its content. Understanding this model demystifies almost every “advanced” Git operation, because they are all just different ways of creating and moving pointers between these objects.

2.2 The Four Object Types



Object	Stores	Key fact
blob	Raw file content only — no filename, no permissions	Two files with identical content share the SAME blob, even in totally different directories
tree	A directory listing: filename → mode + object hash (blob or another tree)	This is how Git represents directory structure
commit	A pointer to one tree (the snapshot), pointer(s) to parent commit(s), author, committer, timestamp, message	A commit is a snapshot , not a diff — Git computes diffs on the fly by comparing trees
tag (annotated)	A named, signable pointer to a commit	Different from a lightweight tag, which is just a ref (Section 10)

```
# Explore the object database directly — this demystifies Git faster than any tutorial
git cat-file -t <hash>           # what TYPE of object is this?
git cat-file -p <hash>          # PRETTY-PRINT its content

# Find the blob hash for a specific tracked file at the current commit
git rev-parse HEAD:path/to/file.txt
git cat-file -p HEAD:path/to/file.txt    # equivalent to `cat file.txt` at that
commit

# See the raw commit object
git cat-file -p HEAD
# tree <hash>
# parent <hash>
# author Dhanush <email> 16900000000 +0530
# committer Dhanush <email> 16900000000 +0530
#
# commit message here
```

WHY COMMITS ARE SNAPSHOTS, NOT DIFFS

This is the single most important mental model correction for intermediate Git users. Git does NOT store “file X changed by these lines” per commit — it stores a full tree snapshot, and reuses unchanged blobs/trees between commits (which is why this is efficient, not wasteful). Diffs shown by `git diff` / `git log -p` are computed on-demand by comparing two snapshots. This is why operations like `git revert` and cherry-picking work the way they do, and why moving a file that has an identical twin elsewhere in the repo costs Git almost nothing.

2.3 The .git Directory

```
ls -la .git/
```

Path	Purpose
<code>.git/objects/</code>	The actual object database (blobs, trees, commits, tags), content-addressed by hash, zlib-compressed
<code>.git/refs/heads/</code>	Branch pointers — each file contains the commit hash a branch currently points to
<code>.git/refs/tags/</code>	Tag pointers
<code>.git/refs/remotes/</code>	Remote-tracking branch pointers (e.g., <code>origin/main</code>)
<code>.git/HEAD</code>	A pointer to the currently checked-out branch (or a raw commit hash, if in detached HEAD state)
<code>.git/index</code>	The staging area — a binary file listing exactly what will go into the next commit
<code>.git/config</code>	Repository-local configuration (remotes, user info overrides, aliases)
<code>.git/logs/</code>	The reflog — a local, personal history of where HEAD and branches have pointed (Section 6.4) — your recovery safety net
<code>.git/packed-refs</code> , <code>.git/objects/pack/</code>	Compressed/packed storage Git uses after <code>git gc</code> for efficiency at scale

```

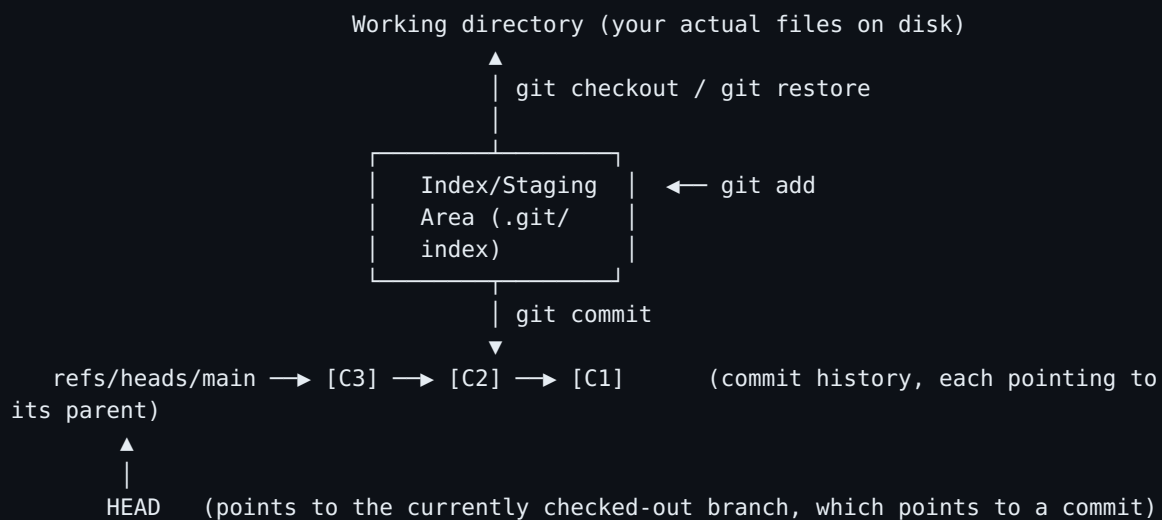
cat .git/HEAD                # ref: refs/heads/main   (normal state)
cat .git/refs/heads/main     # <full commit hash>    – this IS the branch
git symbolic-ref HEAD        # same info, via a proper command instead of
                             # reading the file

```

THE KEY INSIGHT

A “branch” in Git is nothing more than a movable pointer (a 41-byte text file) to a single commit. This is why creating a branch is instant regardless of repo size, and why understanding refs (Section 2.4) unlocks nearly every advanced Git workflow.

2.4 Refs, HEAD, and the Working Tree



Git has **three working areas**: the **working directory** (your actual files), the **staging area / index** (what's queued for the next commit), and the **repository** (committed history, in `.git/objects`). Nearly every confusing Git command is really about moving content between these three areas.

```
git status # the state of all three areas, relative to each other – read this output carefully every time
```

DETACHED HEAD, EXPLAINED

Normally, `HEAD` points to a branch (e.g., `refs/heads/main`), which points to a commit. If you `git checkout <commit-hash>` directly (not a branch name), `HEAD` points DIRECTLY at that commit instead — this is “detached HEAD.” You can look around and even make commits, but no branch is tracking them: if you switch away without creating a branch first, those commits become unreachable (though still recoverable via `reflog` for a while — Section 16.2). Always `git checkout -b temp-branch-name` before committing anything in a detached HEAD state.

3. Git Fundamentals

3.1 Setup and Configuration

```
git config --global user.name "Dhanush"
git config --global user.email "dhanush@example.com"
git config --global init.defaultBranch main          # modern default branch name
git config --global core.editor "vim"                # or "code --wait", "nano",
etc.
git config --global pull.rebase false                 # explicit default pull
strategy (Section 5.4)

git config --list --show-origin                       # see every effective setting AND which file it
came from
git config --global -e                               # edit ~/.gitconfig directly in your editor
```

Configuration has three scopes, in increasing precedence: `--system` (all users on the machine), `--global` (this user, `~/.gitconfig`), `--local` (this repo only, `.git/config` — the default when no flag is given).

3.2 Creating and Cloning Repositories

```
git init myproject                                # create a new, empty repo
git init                                           # initialize the CURRENT directory as a repo

git clone https://github.com/org/repo.git          # HTTPS clone
git clone git@github.com:org/repo.git              # SSH clone (Section 14.2
covers key setup)
git clone --depth 1 https://github.com/org/repo.git # SHALLOW clone – history-
less, much faster for CI checkouts
git clone --branch develop --single-branch <url>   # clone just one branch,
not all of them
```

3.3 The Staging Area: add, status, diff

```
git status                                # what's changed, staged vs unstaged vs untracked
git status -s                             # short/porcelain format – great for scripting:
M=modified, A=added, ??=untracked, etc.

git add file.txt                           # stage a specific file
git add .                                  # stage everything in and below the current
directory
git add -p                                 # PATCH mode – interactively choose which HUNKS to stage, not whole files (essential
# for clean, atomic commits)
git add -u                                 # stage modifications/deletions to already-
tracked files only, not new untracked files

git diff                                   # unstaged changes: working directory vs the index
git diff --staged                          # (or --cached) staged changes: index vs the last commit – "what will actually be
committed"
git diff HEAD                             # working directory vs the last commit (staged
+ unstaged combined)
git diff main..feature                    # compare two branches
git diff --stat                           # summary only: files changed, insertions/
deletions count, no content
```

COMMON MISTAKE

Running `git add .` blindly and committing without reviewing `git diff --staged` first is one of the most common ways secrets, debug code, and unrelated changes end up in a commit. Make `git diff --staged` before every commit a habit, or use `git add -p` to build genuinely intentional, atomic commits.

3.4 Committing

```
git commit -m "Add health check endpoint" # commit with an inline
message
git commit                                # opens your
configured editor for a longer message
git commit -am "Fix typo"
# -a: automatically stage modifications to TRACKED files first (skips untracked files,
and skips `git add` entirely)
git commit --amend                         # rewrite the
MOST RECENT commit (message and/or staged changes) – creates a NEW commit object,
doesn't edit in place
git commit --amend --no-edit
# amend staged changes onto the last commit, keeping its existing message
```

CRITICAL RULE ABOUT -AMEND

`git commit --amend` creates a brand-new commit object with a new hash and abandons the old one — it does not edit history in place. This is safe on commits you haven't pushed yet. Amending (or any history rewrite) on a commit **already pushed and pulled by others** requires a force-push and will break their local history unless coordinated — this is the golden rule covered again in Section 5.5.

3.5 Viewing History

```
git log                # full commit history, newest first
git log --oneline      # one line per commit – hash + subject
git log --oneline --graph --all    # ASCII graph of ALL branches – the single
most useful log view for understanding branch structure
git log -p -- path/to/file.txt    # full diffs, restricted to one file's
history
git log --author="Dhanush"        # filter by author
git log --since="2 weeks ago" --until="yesterday"
git log --grep="JIRA-1234"        # search commit MESSAGES
git log -S "functionName"        # find commits that added/removed a
specific STRING in the code (the "pickaxe" search)
git log --follow -- path/to/renamed_file.txt    # follow history across a file
rename

git show <commit-hash>          # full details + diff of one specific commit
git show <commit-hash>:path/to/file    # the content of a file AT a specific commit

git blame path/to/file.py        # who last changed each line, and in which
commit – essential before "fixing" unfamiliar code
git blame -L 50,80 path/to/file.py    # restrict blame to a line range
```

NOTE

`git log --oneline --graph --all --decorate` is worth aliasing (`git config --global alias.lg "log --oneline --graph --all --decorate"`) — it's the fastest way to actually SEE what a confusing branch/merge situation looks like before deciding what to do about it.

4. Branching & Merging

4.1 What Branching Is and Why It's Cheap

A branch is a movable pointer to a commit (Section 2.4) — creating one is an instant, cheap operation regardless of repository size, because Git isn't copying any files. This is fundamentally different from version control systems (like old-style SVN) where branching meant copying a whole directory tree.

```
git branch                # list local branches, * marks the current one
git branch -a             # list local AND remote-tracking branches
git branch -v             # list with the latest commit on each
git branch feature/login  # create a new branch (does NOT switch to it)
git switch feature/login  # switch to it (modern command, Git 2.23+)
git switch -c feature/login # create AND switch in one step
(equivalent to `git checkout -b`)
git checkout -b feature/login # the older, still very common
equivalent

git branch -d feature/login # delete a branch – SAFE: refuses if it has
unmerged commits
git branch -D feature/login # force-delete – discards unmerged commits,
no warning

git branch -m old-name new-name # rename a branch
```

CHECKOUT VS SWITCH VS RESTORE

Historically, `git checkout` did three unrelated things depending on its arguments: switch branches, restore files, or create a new branch. Git 2.23+ split these into `git switch` (branches) and `git restore` (files) specifically to reduce this ambiguity. Both command families work identically today; `switch` / `restore` are simply clearer to read in scripts and documentation.

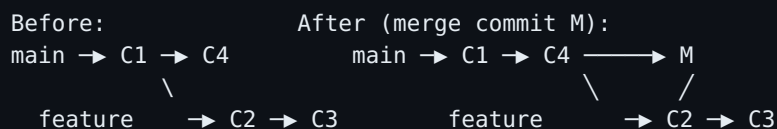
4.2 Merging: Fast-Forward vs Three-Way

```
git switch main
git merge feature/login
```

Fast-forward merge: if `main` has had no new commits since `feature/login` branched off, Git simply moves the `main` pointer forward to `feature/login`'s tip — no new commit is created, and history stays perfectly linear.

```
Before:  main → C1          After (fast-forward):  main → C1 → C2 → C3
          \                                     \      (feature
deleted, main just moved)                       → C2 → C3
          feature
```

Three-way (true) merge: if both branches have diverged (each has commits the other lacks), Git creates a new **merge commit** with **two parents**, combining both histories.



```
git merge --no-ff feature/login      # force a merge commit even when fast-forward is
possible – preserves the fact that a feature branch existed (many teams require this
for auditability)
git merge --squash feature/login      # combine ALL of feature's commits into a
single new commit on main, staged but not yet committed – loses individual commit
history, gains a clean linear main
```

4.3 Merge Conflicts

A conflict happens when the same region of the same file was changed differently on both branches being merged (Git cannot automatically decide which change “wins”).

```
git merge feature/login
# Auto-merging src/app.py
# CONFLICT (content): Merge conflict in src/app.py
# Automatic merge failed; fix conflicts and then commit the result.

git status                # shows which files are in "both modified" / unmerged state
```

```
<<<<<<< HEAD
def login(username, password):
    return authenticate(username, password)
=====
def login(user, pwd):
    return auth_service.check(user, pwd)
>>>>>> feature/login
```

```
# After manually editing the file to resolve the conflict (remove the markers, keep the
intended code):
git add src/app.py          # mark this file's conflict as resolved
git commit                  # completes the merge (message is pre-filled)

# Or abort entirely and go back to the pre-merge state
git merge --abort

# Use a visual merge tool instead of manual markers
git mergetool
```

PRODUCTION PRACTICE

On any file with a non-trivial conflict, don't just make the markers disappear — re-read the ENTIRE resulting function/block, not just the conflicted lines, and run the test suite before committing the merge. A syntactically “resolved” conflict that silently drops one side's logic is one of the most common causes of a regression that slips through review, because the diff of a merge commit is often not shown by default in review tools.

4.4 Merge Strategies on GitHub

Strategy	What it does	When to use
Merge commit	Standard three-way merge, preserves full branch history	Default; good when individual commit history on the feature branch has value
Squash and merge	Combines all PR commits into one commit on the base branch	Keeps <code>main</code> history clean/linear; loses granular commit history (still in the PR itself on GitHub) — very common team default
Rebase and merge	Replays each of the PR's commits individually onto the base branch tip, no merge commit	Keeps a fully linear history AND individual commits; requires clean, well-formed commits on the feature branch

Part of choosing a strategy is a team convention decision (Section 17 covers this as a best practice), not just a technical one — consistency across the team matters more than which specific strategy is “best.”

5. Rebasing

5.1 What It Is and Why It Exists

Rebasing takes the commits on your branch and **replays them one by one on top of a different base commit**, producing a linear history instead of a merge commit. It solves the same problem as merging (integrating divergent work) but produces a fundamentally different-looking history.

Before rebase:

main → C1 → C4

feature ↘
→ C2 → C3

After `git rebase main` (on feature):

main → C1 → C4 → C2' → C3'

(feature now points here)

```
git switch feature/login
git rebase main
# Git replays C2, C3 as brand NEW commits (C2', C3') on top of C4 – same content
changes, DIFFERENT hashes
```

CRITICAL: REBASING REWRITES HISTORY

Every replayed commit gets a NEW hash, because a commit's hash is derived partly from its parent — change the parent, and every descendant commit's hash changes too. This is completely safe for commits that exist ONLY on your local branch, which nobody else has fetched. It is dangerous for commits that have already been pushed and pulled by teammates, because their local history now diverges irreconcilably from yours (Section 5.5, the Golden Rule).

5.2 Interactive Rebase: Cleaning Up History

```
git rebase -i HEAD~4          # interactively edit the last 4 commits on the current
branch
```

```
pick a1b2c3d Add login endpoint
pick e4f5g6h Fix typo in login
pick h7i8j9k Add tests for login
pick k1l2m3n WIP debugging, remove later

# Commands:
# p, pick    = use commit as-is
# r, reword  = use commit, but edit the commit message
# e, edit    = use commit, but STOP for amending (add more changes, split it, etc.)
# s, squash  = combine this commit INTO the previous one, and merge messages
# f, fixup   = like squash, but DISCARD this commit's message entirely
# d, drop    = remove this commit entirely
```



```
# A realistic cleanup before opening a PR: combine "Fix typo" into the commit before
it,
# and drop the leftover debug commit entirely
pick a1b2c3d Add login endpoint
fixup e4f5g6h Fix typo in login
pick h7i8j9k Add tests for login
drop k1l2m3n WIP debugging, remove later
```

PRODUCTION PRACTICE

Interactive rebase before opening a PR — squashing “fix typo” / “address review comment” / “oops” commits into clean, logically atomic commits — makes code review dramatically easier and produces a genuinely useful `git log` for future debugging (e.g., `git bisect`, Section 7.3). Many teams enforce this via a required “squash and merge” GitHub setting instead of relying on individual discipline (Section 4.4).

5.3 Rebase Conflicts

Rebase conflicts are resolved commit-by-commit, not all at once like a merge:

```
git rebase main
# CONFLICT (content): Merge conflict in src/app.py
# (fix the conflict markers in the file)
git add src/app.py
git rebase --continue      # move on to replaying the NEXT commit (may conflict
again)
git rebase --skip
# skip this commit entirely (rare — usually means it's now redundant)
git rebase --abort        # bail out completely, return branch to its pre-
rebase state — always safe
```

5.4 Rebase vs Merge: When to Use Which

	Merge	Rebase
History shape	Preserves true chronological/ branching history, including merge commits	Produces a clean, linear history
Commit hashes	Unchanged	Rewritten for every replayed commit
Safety on shared branches	Always safe	Unsafe on commits others have already pulled
<code>git bisect</code> friendliness	Can be noisier (merge commits can obscure the actual change)	Cleaner, more precise bisecting
Common convention	Merging feature branches INTO shared branches (<code>main</code> , <code>develop</code>)	Rebasing your OWN feature branch onto the latest <code>main</code> before opening/updating a PR

```
git config --global pull.rebase true
# make `git pull` rebase your local commits on top of upstream changes, instead of
creating a merge commit – a common, cleaner-history team default
git pull --rebase                                # do it once, without changing the global
default
```

5.5 The Golden Rule of Rebasing

THE GOLDEN RULE

Never rebase commits that have been pushed to a shared branch and that other people may have already pulled. Rebasing rewrites commit hashes; anyone who has that old history will get confusing divergence, duplicate-looking commits, or broken merges when they next pull. Rebase freely on your own local, not-yet-shared feature branches. If a shared branch's history genuinely must be rewritten (removing a committed secret, for example — Section 16.4), it requires explicit team coordination: everyone re-clones or carefully resets their local copies afterward.

6. Undoing Changes

6.1 The Three Undo Commands and What Each One Touches

Command	Working directory	Staging area (index)	Commit history
<code>git restore <file></code>	Restores file content	Unaffected	Unaffected
<code>git reset --soft <commit></code>	Unaffected	Unaffected	Moves branch pointer; changes from undone commits become STAGED
<code>git reset --mixed <commit></code> (default)	Unaffected	Un-stages everything from undone commits	Moves branch pointer
<code>git reset --hard <commit></code>	Overwritten — uncommitted work is LOST	Cleared to match the target commit	Moves branch pointer
<code>git revert <commit></code>	Updated via a new commit	Updated via a new commit	Adds a NEW commit that undoes the change — original commit stays in history

```
git restore file.txt                # discard uncommitted changes to a file (working dir → matches index/HEAD)
git restore --staged file.txt       # un-stage a file (keeps the actual edits in your working directory)

git reset --soft HEAD~1             # undo the last commit, but keep its changes STAGED – great for "I want to redo this commit's message/contents"
git reset HEAD~1                   # (--mixed, the default) undo the last commit, changes become unstaged but still present in your files
git reset --hard HEAD~1             # undo the last commit AND discard its changes entirely – DESTRUCTIVE

git reset --hard origin/main        # forcibly make your local branch match a remote exactly, discarding local commits/changes – common "just throw away my local mess" recovery move

git revert HEAD                    # create a NEW commit that undoes the most recent commit
git revert <commit-hash>           # revert a specific commit anywhere in history
git revert --no-commit HEAD~3..HEAD # revert a RANGE of commits, staged together as one combined revert (review before committing)
```

THE SINGLE MOST IMPORTANT SAFETY DISTINCTION

`git reset --hard` rewrites your branch pointer and discards data with no new commit created — dangerous on shared branches, since it doesn't create a record of the undo and can lose real work if used carelessly. `git revert` is SAFE on shared/pushed branches: it

adds a new commit on top, so history is preserved and nobody else's clone becomes inconsistent. As a rule: use `reset` only on local, unpushed history; use `revert` to undo anything already shared.

6.2 Amending and Splitting Commits

```
git commit --amend                # edit the message and/or staged content of
the LAST commit

git rebase -i HEAD~3              # mark a commit as "edit" to split it or
add missed changes
# ... rebase stops at that commit ...
git reset HEAD^                  # un-commit it, keeping changes staged/
unstaged for you to split
git add -p                      # stage part of the changes
git commit -m "First half of the split"
git add .
git commit -m "Second half of the split"
git rebase --continue
```

6.3 Recovering Lost Work: The Reflog

Even after a `reset --hard` or a bad rebase, the commits you “lost” almost always still exist in Git’s object database — they’re just unreferenced by any branch. The **reflog** is your local, personal log of everywhere `HEAD` has pointed, and it’s the recovery tool of last resort.

```
git reflog                        # a chronological log of every HEAD movement on
THIS machine: commits, resets, rebases, checkouts
# a1b2c3d HEAD@{0}: reset: moving to HEAD~1
# e4f5g6h HEAD@{1}: commit: Add login validation    ← this is the "lost" commit
# h7i8j9k HEAD@{2}: checkout: moving from main to feature/login

git checkout e4f5g6h              # go look at the lost commit directly
git branch recovered-work e4f5g6h # create a new branch pointing at it – fully
recovered
```

NOTE

The reflog is LOCAL ONLY — it is never pushed and doesn’t exist on GitHub or for anyone else’s clone; it only helps recover from your own local mistakes. It also expires (default: unreachable entries after 30 days, reachable ones after 90 — configurable via `gc.reflogExpire`). This is precisely why “I accidentally hard-reset and lost my work” is recoverable in the vast majority of real cases, and one of the most valuable things to know cold for both real incidents and interviews.

6.4 Cleaning Untracked Files

```
git clean -n          # DRY RUN – show what would be deleted, without deleting
                        anything (always run this first)
git clean -f          # actually delete untracked files
git clean -fd         # also delete untracked DIRECTORIES
git clean -fx         # also delete files ignored by .gitignore (e.g., wipe
                        build artifacts/node_modules entirely)
```

SAFETY HABIT

`git clean` permanently deletes files that were never committed — the reflog CANNOT help you recover from this, because reflog only tracks committed history. Always run `git clean -n` first to review exactly what will be removed.

7. Stashing, Cherry-Picking, and Bisecting

7.1 git stash: Temporary Shelving of Changes

```
git stash                                # shelve all uncommitted changes (staged +
unstaged), restore a clean working directory
git stash push -m "WIP: login refactor"  # stash with a descriptive message
git stash push -- path/to/file.txt      # stash only specific files
git stash -u                            # also stash untracked files (not
included by default)

git stash list                          # see all stashes: stash@{0}, stash@{1}, ...
git stash show -p stash@{0}            # view the actual diff in a stash

git stash pop                          # apply the most recent stash AND remove it from
the stash list
git stash apply stash@{1}
# apply a specific stash but KEEP it in the list (useful to apply the same stash to
multiple branches)
git stash drop stash@{0}                # delete a stash without applying it
git stash clear                        # delete ALL stashes
```

PRODUCTION SCENARIO

You're mid-way through a feature when an urgent production bug comes in. `git stash` your in-progress work, `git switch main`, branch off for the hotfix, fix and ship it, then `git switch` back to your feature branch and `git stash pop` to resume exactly where you left off — without committing half-finished work just to switch context.

7.2 git cherry-pick: Applying Specific Commits

```
git cherry-pick <commit-hash>          # apply ONE specific commit's changes onto
the current branch, as a new commit
git cherry-pick <hash1> <hash2>        # apply several specific commits, in order
git cherry-pick <hashA>^..<hashB>      # apply a RANGE of commits (exclusive of
hashA, inclusive of hashB)
git cherry-pick --no-commit <hash>     # apply the changes but don't commit
yet — lets you combine/adjust first
git cherry-pick --continue              # after resolving a cherry-pick
conflict
git cherry-pick --abort                 # bail out
```

PRODUCTION SCENARIO: HOTFIX BACKPORTING

A critical bugfix was committed to `main`, but you also need it on a maintenance branch (`release/2.4`) that doesn't include all of `main`'s other recent changes. `git cherry-pick <fix-commit-hash>` while on the release branch applies just that one change, cleanly, without merging in everything else from `main`.

7.3 git bisect: Binary-Searching for a Regression

When you know a bug exists now but not exactly which commit introduced it, `bisect` automates a binary search across history.

```
git bisect start
git bisect bad           # the CURRENT commit is known bad
git bisect good v1.4.0   # this earlier tag/commit is known good

# Git checks out a commit roughly halfway between good and bad
# Test it, then tell Git the result:
git bisect good          # or
git bisect bad
# Git narrows the range and checks out the next candidate – repeat until Git identifies
the exact culprit commit

git bisect reset        # when done, return to your original branch/commit
```

```
# Automate the whole search with a script that exits 0 (good) or non-zero (bad)
git bisect start HEAD v1.4.0
git bisect run ./run_tests.sh
# Git automatically checks out, runs the script, and narrows the range with NO manual
interaction
```

PRODUCTION SCENARIO

“Something between last Tuesday’s deploy and today broke checkout, but nobody knows which of the 40 merged PRs did it.” `git bisect start`, mark today as bad and last Tuesday’s release tag as good, then either manually test each candidate commit or point `git bisect run` at an automated test/reproduction script. On a 40-commit range, this finds the exact culprit in about 6 steps ($\log_2 40 \approx 5.3$) instead of manually checking commits one by one.

8. Remote Repositories & Collaboration

8.1 Remotes

A **remote** is just a named URL pointing to another copy of the repository — **origin** by convention for the one you cloned from, but there can be several (e.g., **upstream** for a fork's original source).

```
git remote -v                # list remotes and their URLs (fetch + push)
git remote add upstream https://github.com/original-org/repo.git
git remote set-url origin git@github.com:org/repo.git    # switch an existing remote
from HTTPS to SSH
git remote remove upstream
git remote show origin      # detailed info: tracked branches, HEAD
branch, push/pull config
```

8.2 fetch vs pull vs push

```
git fetch origin
# download new commits/branches from the remote – does NOT touch your working directory
or current branch
git fetch --all              # fetch from every configured remote

git pull
# fetch + immediately merge (or rebase, if configured) into your current branch
git pull --rebase            # fetch + rebase your local commits on top instead
of merging
git pull origin main --ff-only # only allow a fast-forward pull; fail loudly
instead of creating a surprise merge commit – a safer default for automation/scripts

git push                    # upload your local commits to the remote's tracking
branch
git push -u origin feature/login # push AND set up tracking (so future plain `git
push`/`git pull` know the mapping) – do this once per new branch
git push origin --delete feature/login # delete a branch on the remote
```

FETCH VS PULL, IN ONE SENTENCE

fetch updates your knowledge of the remote (safe, non-destructive, never touches your working files); **pull** is **fetch** + an automatic integration (merge or rebase) into your current branch. When in doubt or reviewing someone else's changes before integrating, prefer **fetch** followed by an explicit **git log origin/main / git diff main origin/main** review, then merge or rebase deliberately.

8.3 Tracking Branches and Force-Pushing Safely

```
git branch -vv                # show each local branch's upstream tracking
                              # branch and ahead/behind counts
git branch --set-upstream-to=origin/feature/login    # manually link a local branch to
a remote one

git push --force              # DANGEROUS – overwrites the remote branch
unconditionally, can silently discard others' pushed commits
git push --force-with-lease   # SAFER – refuses the push if the remote branch
has commits you haven't fetched yet (i.e., someone else pushed since you last synced)
```

PRODUCTION STANDARD: ALWAYS `--force-with-lease`, NEVER BARE `--force`

After an interactive rebase or amend on a branch you've already pushed (your own feature branch, never a shared branch like `main`), you must force-push to update the remote. `--force-with-lease` checks that the remote hasn't changed since your last fetch before overwriting it — it protects you from clobbering a teammate's commits that you simply didn't know about yet. Many teams disable plain `git push --force` via a pre-push hook or branch protection rule and require `--force-with-lease` as the only allowed form.

8.4 Forking Workflow

```
# Standard open-source / cross-team contribution pattern:
# 1. Fork the repo on GitHub (creates your own copy under your account)
git clone git@github.com:yourname/repo.git    # clone YOUR fork
git remote add upstream git@github.com:original-org/repo.git
# track the original as "upstream"

# 2. Keep your fork's main in sync with upstream
git fetch upstream
git switch main
git merge upstream/main                      # or rebase, depending on team convention
git push origin main                         # update your fork on GitHub

# 3. Work on a feature branch, push to YOUR fork, open a PR from your fork to upstream
git switch -c feature/new-thing
# ... commits ...
git push -u origin feature/new-thing
# then open a PR on GitHub: yourname/repo:feature/new-thing → original-org/repo:main
```

9. Git Branching Strategies

9.1 Why This Matters

A branching strategy is a team-wide convention for how work moves from “in progress” to “in production.” Picking the wrong one for your team’s size, release cadence, and deployment model creates constant friction — long-lived branches that are painful to merge, unclear “what’s actually in production right now,” or a review process that can’t keep up. This is one of the most commonly asked practical/architectural questions in DevOps interviews.

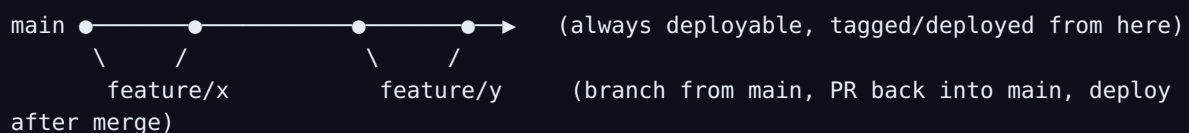
9.2 Trunk-Based Development



Everyone works off very short-lived branches (hours to a couple of days) merged frequently into a single `main` / `trunk`, which is always kept deployable. Feature flags gate incomplete work rather than long-lived branches. This is the dominant model at high-velocity, CI/CD-mature organizations (paired with automated testing and often continuous deployment).

- **Pros:** minimal merge conflict pain, fast feedback, forces small PRs, `main` always reflects near-current reality.
- **Cons:** requires strong CI/test discipline and often feature flags for incomplete work; risky without good automated test coverage.

9.3 GitHub Flow



A simplified model: branch off `main` for any change, open a PR, review, merge back into `main`, deploy. No `develop` branch, no complex release branches. Very close to trunk-based development but with slightly longer-lived feature branches and a stronger PR-centric review culture. This is GitHub’s own recommended default and fits most web-application teams well.

9.4 Git Flow



A heavier, more structured model with long-lived **main** (production) and **develop** (integration) branches, plus dedicated **feature/***, **release/***, and **hotfix/*** branch types with defined merge rules between them.

- **Pros:** clear separation between “in development” and “released,” good for scheduled/versioned releases (e.g., mobile apps, installed software, or anything with a formal release train).
- **Cons:** heavier process, more merge overhead, long-lived **develop** can drift significantly from **main**; poor fit for continuous deployment.

9.5 Choosing a Strategy

Team/product profile	Good fit
Web service, continuous deployment, strong test automation	Trunk-based development or GitHub Flow
Mobile app, installed software, scheduled/versioned releases	Git Flow (or a lighter variant with just release/* branches)
Small team, simple product, frequent small PRs	GitHub Flow
Regulated environment needing a clear release/hotfix audit trail	Git Flow, or GitHub Flow + protected release tags

INTERVIEW-RELEVANT FRAMING

Don’t just name the strategies — be ready to explain WHY a given team would choose one: deployment frequency, release cadence, team size, and how much safety net (test automation, feature flags) exists to make short-lived trunk-based branches safe. This “it depends, and here’s the tradeoff” answer is what distinguishes real production judgment from memorized definitions.

10. Tags & Releases

10.1 Lightweight vs Annotated Tags

```
git tag v1.4.0 # LIGHTWEIGHT tag – just a named pointer to a
commit, no extra metadata
git tag -a v1.4.0 -m "Release 1.4.0: add SSO support" # ANNOTATED tag – a full Git
object with tagger, date, message, and can be GPG-signed

git tag -a v1.4.0 -m "..." <commit-hash> # tag a specific (not necessarily HEAD) c
ommit

git tag # list all tags
git tag -l "v1.4.*" # filter by pattern
git show v1.4.0 # view an annotated tag's message + the
commit it points to

git push origin v1.4.0 # tags are NOT pushed automatically – push
explicitly
git push origin --tags # push all local tags at once
git tag -d v1.4.0 # delete a local tag
git push origin --delete v1.4.0 # delete a tag on the remote
```

PREFER ANNOTATED TAGS FOR RELEASES

Always use annotated tags (`-a`) for anything representing an actual release — they carry a tagger name, date, and message as a real Git object, and are what GitHub Releases are built from. Lightweight tags are fine for quick, personal, throwaway markers.

10.2 Semantic Versioning

Most tagged releases follow **SemVer**: `MAJOR.MINOR.PATCH` (e.g., `v2.4.1`).

Segment	Increment when
MAJOR	Breaking/incompatible API changes
MINOR	New backward-compatible functionality
PATCH	Backward-compatible bug fixes only

This convention matters far beyond Git itself — it's what Terraform module version constraints, npm/pip dependency ranges, and Helm chart versions (later parts) all assume.

11. .gitignore, .gitattributes, and Git LFS

11.1 .gitignore

```
# .gitignore – patterns for files Git should never track
node_modules/
*.log
.env
.env.*
!.env.example      # negation: DO track this specific file even though .env.* is
                    # ignored
dist/
build/
*.pyc
__pycache__/
.DS_Store
*.tfstate
*.tfstate.backup
.terraform/
```

```
git check-ignore -v path/to/file      # debug WHY a file is being ignored, and which
rule/line matched
git rm -r --cached node_modules/      # stop tracking a directory that was
accidentally committed BEFORE it was added to .gitignore (removes from Git, keeps it on
disk)
```

COMMON MISTAKE

Adding a pattern to `.gitignore` does nothing for files ALREADY tracked by Git — you must explicitly `git rm --cached` them first. This is the single most common reason “.gitignore isn’t working” reports turn out to be user error, not a Git bug.

11.2 .gitattributes

```
# .gitattributes – per-path Git behavior overrides
*.sh text eol=lf          # force LF line endings for shell scripts even on
Windows checkouts – critical: CRLF breaks shebang-based script execution
*.png binary              # explicitly mark as binary (skip line-ending/
diff processing)
*.pdf diff=off            # don't attempt to diff PDFs
CHANGELOG.md merge=union  # on merge conflicts in this file, keep BOTH
sides' additions instead of conflict-marking (good for append-only logs)
docs/** linguist-documentation # tell GitHub's language-detection to not
count this toward the repo's "language" stats
```

11.3 Git LFS (Large File Storage)

Git is fundamentally bad at large binary files — every version of a binary is stored in full (no meaningful diffing), so repository size balloons quickly. **Git LFS** replaces large files in the repo with small text pointers, storing the actual content in separate LFS storage.

```
git lfs install                # one-time setup per machine
git lfs track "*.psd"          # register a pattern to be LFS-managed
                               # (writes to .gitattributes)
git lfs track "assets/videos/**"
git add .gitattributes
git add large_design_file.psd
git commit -m "Add design assets via LFS"
git push

git lfs ls-files              # see which files are LFS-tracked in this repo
git lfs pull                  # explicitly fetch the actual LFS content (a
                               # plain clone only gets pointers by default in some configs)
```

WHEN TO REACH FOR LFS

Use LFS for large, frequently-changing binaries genuinely needed in version control: design assets, ML model checkpoints, large fixture/test data files. Don't use Git (even with LFS) as a general artifact store for build outputs, container images, or release binaries — those belong in a proper artifact registry (container registries, S3, Artifactory) as covered in later parts.

12. Git Hooks & Automation

12.1 What Hooks Are and Why They Exist

Git hooks are scripts Git automatically runs at specific points in its workflow (before a commit, before a push, after a merge, etc.) — they live in `.git/hooks/` and let you enforce standards or trigger automation locally, without needing a CI server round-trip for fast feedback.

12.2 Common Hooks

Hook	Fires	Typical use
<code>pre-commit</code>	Before a commit is created	Lint/format code, block commits with obvious issues, run fast unit tests
<code>commit-msg</code>	After the commit message is written, before finalizing	Enforce a commit message format (e.g., Conventional Commits, JIRA ticket reference)
<code>pre-push</code>	Before <code>git push</code> sends data	Run a fuller test suite, block pushes to protected branch names
<code>post-checkout</code>	After <code>git checkout / switch</code>	Auto-install dependencies if <code>package.json</code> / <code>requirements.txt</code> changed
<code>post-merge</code>	After a successful merge/pull	Same — refresh dependencies automatically

```
#!/usr/bin/env bash
# .git/hooks/pre-commit (must be executable: chmod +x)
set -euo pipefail

echo "Running pre-commit checks..."

# Block commits containing obvious secret patterns
if git diff --cached | grep -E "(AKIA[0-9A-Z]{16}|-----BEGIN (RSA|OPENSSH) PRIVATE
KEY-----)"; then
    echo "ERROR: Possible secret detected in staged changes. Commit blocked." >&2
    exit 1
fi

# Run a linter against staged files only
files=$(git diff --cached --name-only --diff-filter=ACM -- '*.py')
if [[ -n "$files" ]]; then
    flake8 $files || { echo "Lint failed." >&2; exit 1; }
fi

echo "Pre-commit checks passed."
```

CRITICAL LIMITATION: HOOKS ARE NOT VERSION-CONTROLLED OR ENFORCED BY DEFAULT

`.git/hooks/` is local to each clone and is NOT part of the repository's tracked content — a hook you write is never automatically shared with teammates or enforced in CI. Relying on local hooks alone for anything security-critical (like secret scanning) is unsafe; always back them up with the same checks running in CI/CD (Part 04) as the actual enforcement layer. Hooks are a fast local convenience, not a security control.

12.3 Sharing Hooks Across a Team: pre-commit Framework

```
pip install pre-commit
```



```
# .pre-commit-config.yaml – checked into the repo, so it's version-controlled and
shareable
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.5.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: detect-private-key
  - repo: https://github.com/psf/black
    rev: 24.1.1
    hooks:
      - id: black
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.1
    hooks:
      - id: gitleaks                # secret scanning as a local pre-commit gate
```

```
pre-commit install                # installs the actual .git/hooks/pre-commit that runs
this config
pre-commit run --all-files        # run against the whole repo once (e.g., to check
historical files, or in CI as a backstop)
```

This is the standard modern approach: hooks are declared as version-controlled YAML, every engineer runs one setup command, and the exact same config can run in CI as a second enforcement layer.

12.4 Server-Side Hooks and GitHub Equivalents

Git also supports server-side hooks (`pre-receive` , `update` , `post-receive`) that run on the Git server itself — but GitHub doesn't expose raw server-side hooks to repository owners. Instead, the equivalent enforcement happens through **branch protection rules**, **required status checks**, and **GitHub Actions** (Section 14.4, and Part 04 in full depth) — these are the actual, unavoidable enforcement points, since local hooks can always be skipped with `--no-verify` .

```
git commit --no-verify -m "..."    # bypasses pre-commit/commit-msg hooks entirely –
this is exactly why local hooks alone are not a security boundary
```

13. Submodules and Monorepo vs Polyrepo

13.1 Git Submodules

A submodule embeds one Git repository inside another as a specific, pinned commit — useful for including a shared library or dependency as source, with explicit version control over exactly which commit is used.

```
git submodule add https://github.com/org/shared-lib.git libs/shared-lib
git commit -m "Add shared-lib as a submodule"

# Cloning a repo WITH submodules requires an extra step – this trips people up
constantly
git clone --recurse-submodules https://github.com/org/main-repo.git
# or, if already cloned without it:
git submodule update --init --recursive

# Update a submodule to a newer commit of the sub-repo
cd libs/shared-lib
git pull origin main
cd ../../
git add libs/shared-lib          # stages the submodule's pointer moving to the
new commit
git commit -m "Bump shared-lib to latest"
```

COMMON PAIN POINTS

Submodules are a frequent source of confusion: a plain `git clone` leaves submodule directories EMPTY unless `--recurse-submodules` is used; a submodule pointer records an EXACT commit, so forgetting to commit the updated pointer after pulling inside the submodule means teammates stay on the old version; and CI pipelines must be explicitly configured to check out submodules too. Many teams actively avoid submodules for these reasons, preferring a proper package manager/artifact registry for shared code instead.

13.2 Monorepo vs Polyrepo

	Monorepo (one repo, many projects/services)	Polyrepo (one repo per project/service)
Cross-project changes	Single atomic commit/PR across everything	Requires coordinated changes/ PRs across multiple repos
Tooling	Needs investment: selective CI (only build what changed), code ownership tooling	Simpler per-repo tooling, but harder org-wide visibility
Dependency versioning	Usually “always latest” internally — no version skew between internal libs	Explicit versioning needed between repos (package registries, submodules)
Access control granularity	Coarser by default (though path-based permissions exist on some platforms)	Fine-grained naturally, per repo
Scaling concern	Repo size / CI time at very large scale (Google, Meta use custom tooling for this)	Repo sprawl — hundreds of small repos become hard to track/govern
Real-world examples	Google, Meta, many mid-size platform teams for tightly coupled services	Most open-source projects, teams with clearly independent services

INTERVIEW-RELEVANT FRAMING

There’s no universally “correct” answer — the real skill is being able to articulate the tradeoff for a SPECIFIC situation: a platform team building 5 tightly-coupled microservices that always deploy together leans monorepo (atomic cross-service changes, shared CI); a company offering 20 independent open-source tools leans polyrepo (independent versioning, independent contributor access). Frame your answer around coupling and deployment cadence, not a blanket preference.

14. GitHub Platform Deep Dive

14.1 What GitHub Adds on Top of Git

Git itself has no concept of “pull requests,” “issues,” “branch protection,” or “actions” — these are entirely GitHub platform features built around a plain Git repository. Understanding this separation matters: the same Git skills apply on GitLab, Bitbucket, or a self-hosted server, while the platform-specific collaboration features (this section) are GitHub’s particular implementation of code review and project management.

14.2 Authentication: SSH vs HTTPS vs Tokens

```
# SSH (recommended for regular development use)
ssh-keygen -t ed25519 -C "dhanush@example.com"
cat ~/.ssh/id_ed25519.pub
# paste this into GitHub → Settings → SSH and GPG keys
ssh -T git@github.com # test: should greet you by username

# HTTPS with a Personal Access Token (PAT) – required since GitHub deprecated password
# auth for Git operations
git clone https://github.com/org/repo.git
# Username: your GitHub username
# Password: a Personal Access Token (Settings → Developer settings → Personal access
# tokens), NOT your account password

git config --global credential.helper store # cache credentials after first use
(or use "cache" for a time-limited in-memory cache)
```

Auth method	Good for
SSH key	Daily developer use — no token expiry management, works across many repos with one key
Personal Access Token (classic or fine-grained)	HTTPS clones, scripts/automation, CI runners without SSH agent forwarding
GitHub App / Installation token	Automation/bots acting on behalf of an org, with narrow, auditable, auto-expiring scopes — the production-grade choice for CI/CD integrations (Part 26 covers this in the CI/CD security context)

SECURITY NOTE

Fine-grained PATs (scoped to specific repos and specific permissions, with a mandatory expiry) are strongly preferred over classic PATs (broad, account-wide scope, optional expiry) for anything beyond personal daily use. Never commit a token to a repository —

Section 16.4 covers exactly what to do if this happens, and GitHub’s built-in secret scanning (Section 15.3) will often catch and auto-revoke known token formats within minutes.

14.3 Pull Requests and Code Review

```
# Typical flow
git switch -c feature/add-caching
# ... commits ...
git push -u origin feature/add-caching
# Open a PR on GitHub: base = main, compare = feature/add-caching
```

Key PR mechanics worth knowing precisely:

- **Draft PRs** signal “not ready for review yet” — CI still runs, but reviewers aren’t notified to review.
- **Requested reviewers** vs **CODEOWNERS**-triggered reviewers (Section 14.5) — the latter is automatic and enforceable.
- **Review states:** Comment (no verdict), Approve, Request changes (blocks merge if the branch protection rule requires resolving it).
- **Suggested changes:** a reviewer can propose an exact diff inline, which the PR author can accept with one click, creating a commit automatically.
- A PR is fundamentally a comparison between two refs (`base...compare`) plus a place to attach review/CI state — merging it performs one of the three strategies from Section 4.4.

14.4 Branch Protection Rules

Configured under repo Settings → Branches, these are the actual server-side enforcement mechanism (since GitHub has no raw server-side hooks exposed to users):

Rule	Effect
Require a pull request before merging	Blocks direct pushes to the protected branch entirely
Require approvals (N reviewers)	PR cannot merge until N approvals are given
Require status checks to pass	PR cannot merge until named CI checks (Part 04) report success
Require branches to be up to date before merging	Forces re-testing against the LATEST base branch state before merge — prevents a “worked in isolation, broke combined” surprise
Require conversation resolution	All PR review comment threads must be marked resolved
Require signed commits	Enforces GPG/SSH commit signing (Section 15.2) on this branch
Restrict who can push	Limits direct push rights even further, e.g., to release-engineering only
Do not allow bypassing the above settings	Closes the “admins can override” loophole — important for genuinely enforced compliance

PRODUCTION PRACTICE

A production-grade `main` branch protection setup, at minimum: require PR + at least 1 (often 2) approvals, require the CI status checks that matter (build, tests, security scan) to pass, require branches to be up to date, and disallow force-pushes and direct pushes entirely — including for admins, unless there’s a documented emergency-bypass process.

14.5 CODEOWNERS

```
# .CODEOWNERS (or CODEOWNERS, or docs/CODEOWNERS – GitHub checks all three locations)
* @platform-team
/infrastructure/ @devops-team
/services/payments/ @payments-team @security-team
*.tf @devops-team
/docs/ @tech-writers
```

When a PR touches files matching a pattern, GitHub automatically requests review from the matching owner(s), and (combined with a “require review from Code Owners” branch protection rule) can make that review mandatory — not just a notification.

14.6 Issues, Labels, Milestones, and Projects

- **Issues:** trackable units of work, discussion threads, bug reports — linkable to PRs (`Closes #42` in a PR/commit description auto-closes the issue on merge).

- **Labels:** categorization (`bug` , `priority:high` , `needs-triage`) — filterable, and usable in automation (GitHub Actions can react to label changes).
- **Milestones:** group issues/PRs toward a target date or release.
- **GitHub Projects:** a Kanban/table/roadmap view built on top of issues and PRs across one or more repos, with custom fields — GitHub's built-in lightweight project management layer.

14.7 GitHub CLI (gh)

The `gh` CLI lets you drive the entire GitHub workflow — PRs, issues, releases, Actions runs — without leaving the terminal, which matters heavily for scripting and automation.

```
gh auth login                # authenticate the CLI (browser or token flow)

gh repo clone org/repo      # clone with gh instead of plain git
gh repo create my-new-repo --private  # create a new repo from the CLI

gh pr create --title "Add caching layer" --body "Implements LRU cache for..." --base main
gh pr list                  # list open PRs
gh pr view 42 --web         # open PR #42 in the browser
gh pr checkout 42           # check out a PR's branch locally,
even from a fork
gh pr review 42 --approve -b "LGTM"      # approve a PR from the CLI
gh pr merge 42 --squash --delete-branch  # merge and clean up in one
command

gh issue create --title "Bug: login fails on Safari" --label bug
gh issue list --label "priority:high"

gh run list                 # recent GitHub Actions workflow runs
gh run watch                # live-follow the current run
gh run view <run-id> --log-failed      # pull failed step logs directly into the
terminal – very fast CI debugging loop

gh api repos/org/repo/branches/main/protection
# hit the raw GitHub API for anything not covered by a dedicated subcommand
```

WHY THIS MATTERS FOR AUTOMATION

`gh` is scriptable Bash-friendly output by default and JSON output via `-json` , making it the natural tool for writing operational scripts around GitHub (auto-labeling stale PRs, generating release notes, bulk-closing issues) without hand-rolling REST API calls — and it's usually pre-authenticated inside GitHub Actions runners via the built-in `GITHUB_TOKEN` .

15. Git & GitHub Security

15.1 What It Is and Why It Exists

Git repositories are a high-value target: they hold source code, history (including anything ever committed, even if later “removed”), and increasingly, the automation identity (CI tokens, deploy keys) that touches production. Git/GitHub security is foundational to the supply-chain security topics covered in depth in Part 26, but the basics belong here, at the point where you’re actually using Git day to day.

15.2 Commit Signing (GPG / SSH)

Signing proves a commit or tag was genuinely created by the holder of a specific private key — protecting against commit spoofing (setting `user.name` / `user.email` to impersonate someone is trivial; a cryptographic signature is not).

```
# GPG signing
gpg --full-generate-key                # generate a new GPG keypair
gpg --list-secret-keys --keyid-format=long  # find your key ID
git config --global user.signingkey <KEY-ID>
git config --global commit.gpgsign true    # sign EVERY commit automatically
git commit -S -m "Signed commit"          # sign one commit explicitly
(if not auto-signing)

# SSH signing (simpler — reuses your existing SSH key, Git 2.34+)
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
```

```
git log --show-signature -1    # verify a commit's signature locally
```

On GitHub, a verified commit/tag shows a green “Verified” badge — but this only proves the signature is cryptographically valid and matches a key registered to that account, not that the commit’s *content* is trustworthy. Combine with branch protection’s “Require signed commits” rule to make this an enforced requirement, not just a nice-to-have.

15.3 Secret Scanning and Prevention

```
# Local: gitleaks – scan a repo (or specific commits) for known secret patterns
gitleaks detect --source . -v
gitleaks protect --staged -v          # scan ONLY staged changes – ideal as a pre-
commit hook (Section 12.3)

# Local: trufflehog – similar, with entropy-based detection for less-obvious secrets
trufflehog git file://. --since-commit HEAD~20
```

GitHub itself runs **secret scanning** automatically on public repos (and on private repos with GitHub Advanced Security or via org-level enablement), detecting known token formats (AWS keys, GitHub tokens, common API key patterns) and, for many providers, automatically notifying the provider to revoke the leaked credential — often within minutes, faster than most humans would react.

```
# Enable via API/CLI for a repo you administer
gh api -X PATCH repos/org/repo -f security_and_analysis[secret_scanning][status]=enabled
```

THE MOST IMPORTANT FACT ABOUT LEAKED SECRETS

Once a secret is committed and pushed, treat it as compromised IMMEDIATELY, even if you delete it in the very next commit. The old value remains permanently in Git history (and possibly already scraped by bots — public repos are actively monitored). Deleting the file does NOT remove it from history; the ONLY correct fix is: (1) rotate/revoke the actual credential at its source immediately, THEN (2) optionally clean history if needed for other reasons (Section 16.4). Rotation, not history rewriting, is the real fix.

15.4 Deploy Keys vs Personal Access Tokens vs GitHub Apps

Mechanism	Scope	Best for
Deploy key	One specific repo, read-only by default (optionally read-write)	A single server/service that needs to pull ONE repo (e.g., a deploy target pulling its own config repo)
Personal Access Token	Tied to a human user's permissions, broad by default (esp. classic PATs)	Personal scripting/CLI use; NOT ideal for shared automation (tied to one person's identity, breaks if they leave/get offboarded)
GitHub App / installation token	Narrow, explicitly granted permissions, org-installable, auto-expiring short-lived tokens	The production-correct choice for CI/CD bots, integrations, and any automation acting "as an app," not as a person

COMMON ANTI-PATTERN

Using a personal PAT (often belonging to whoever happened to set up the pipeline) as a long-lived CI secret is a very common but fragile and risky pattern: it's over-scoped to that person's full account access, breaks silently when they leave or rotate their password, and provides poor audit trails ("who did this deploy" shows as a person, not the actual automation). Prefer a GitHub App or the repo's built-in, automatically-scoped

`GITHUB_TOKEN` (covered in Part 04) wherever the use case allows it.

15.5 Repository-Level Security Settings Checklist

```
gh api repos/org/repo | jq '.security_and_analysis' # inspect current security
feature status via the CLI
```

- **Secret scanning + push protection** (blocks a push containing a detectable secret BEFORE it ever lands in history — strictly better than after-the-fact detection)
- **Dependabot alerts** and **Dependabot security updates** (automated PRs for vulnerable dependency versions — deep-dived in Part 26)
- **Code scanning** (CodeQL or a third-party SAST tool wired into Actions — also Part 26)
- Branch protection on `main` (and any release branches) per Section 14.4
- Restrict who can create/delete branches matching protected patterns
- Review and prune stale collaborator access and unused deploy keys/tokens periodically

16. Troubleshooting Git

16.1 A Systematic Approach

Git troubleshooting almost always reduces to answering one question first: **which of the three areas (working directory, staging area, commit history — Section 2.4) actually holds the state I want, and which command moves it to where I need it?** Resist the urge to guess-and-check random commands; `git status` and `git log --oneline --graph --all` first, always.

16.2 Scenario: “I accidentally hard-reset and lost commits”

```
git reflog                # find the commit hash from before the reset
git branch recovered <hash> # create a branch pointing at it – fully recovered,
                           # nothing was actually deleted
```

See Section 6.3 for the full explanation of why this works.

16.3 Scenario: Merge conflict I can't figure out

```
git status                # which files are actually conflicted right now
git diff                  # see the conflict markers in context
git log --merge --oneline # commits that are relevant to the current
conflicted merge, from BOTH sides
git show :1:path/to/file  # the common ANCESTOR version of the file
(stage 1)
git show :2:path/to/file  # "ours" – the current branch's version
(stage 2)
git show :3:path/to/file  # "theirs" – the branch being merged in
(stage 3)

# When truly stuck, it's always safe to bail and start over:
git merge --abort
```

16.4 Scenario: A secret was committed and needs to be purged from history

```
# STEP 1 – always first, non-negotiable: rotate/revoke the actual credential at its
source (Section 15.3)

# STEP 2 (optional, only if history-cleanup is still required for compliance/hygiene) –
use git-filter-repo (the modern, fast, officially-recommended tool; NOT the old filter-
branch)
pip install git-filter-repo
git filter-repo --path secrets/config.env --invert-paths          # remove a file from
ALL history

# Or scrub a specific STRING wherever it appears across history
git filter-repo --replace-text <(echo "AKIAABCDEFGHIJKLMNOP==>REDACTED")

# STEP 3 – force-push the rewritten history and have EVERY collaborator re-clone (their
old history is now permanently incompatible)
git push origin --force --all
git push origin --force --tags
```

WHY ROTATION ALWAYS COMES FIRST

History rewriting does nothing to protect you from a secret that was already fetched by a bot, cached by a CI system, or forked by someone else before you noticed — those copies are completely unaffected by rewriting YOUR repo's history. Rotating the actual credential is the only action that genuinely closes the exposure; history cleanup is purely a hygiene/compliance step afterward, not a security fix by itself.

16.5 Scenario: Accidentally committed to the wrong branch

```
# You committed directly to main, meant to be on a feature branch
git branch feature/oops          # create a new branch AT the current (wrong)
position – captures the commit
git reset --hard origin/main
# move main back to match the remote, discarding the local commit FROM main (safe: it's
preserved on feature/oops now)
git switch feature/oops          # continue your work on the correctly-named
branch
```

16.6 Scenario: Detached HEAD with commits you want to keep

```
git status          # confirms "HEAD detached at <hash>" and often
already suggests this
git switch -c rescue-branch
# create a branch AT the current detached position – commits are now safely referenced
```

16.7 Scenario: Diverged branches (“your branch and ‘origin/main’ have diverged”)

```
git status
# Your branch and 'origin/main' have diverged, and have 2 and 3 different commits each,
respectively.

git log --oneline --graph --all --left-right main...origin/main    # visualize exactly
what differs on each side

# Option A – you want your local commits ON TOP of the latest remote history (linear,
if unpushed/uncoordinated)
git pull --rebase

# Option B – you want a genuine merge preserving both histories (safer default for
already-shared branches)
git pull --no-rebase
```

16.8 Diagnostic Command Reference

Symptom	Diagnostic command
“What state am I actually in?”	<code>git status</code>
“What does history actually look like?”	<code>git log --oneline --graph --all</code>
“Where has HEAD been, including ‘lost’ commits?”	<code>git reflog</code>
“What exactly changed in this commit?”	<code>git show <hash></code>
“Which commit introduced this line/bug?”	<code>git blame</code> , <code>git log -S</code> , <code>git bisect</code>
“What’s different between my branch and main?”	<code>git diff main...HEAD</code> (three dots: from the merge-base, not just main’s tip)
“Why is this file being ignored/not ignored?”	<code>git check-ignore -v <file></code>

17. Best Practices and Anti-Patterns

17.1 Commit Hygiene

GOOD COMMIT PRACTICES

- **Atomic commits:** each commit represents one logical change, and the codebase builds/tests pass at every single commit — not just at the tip of the branch. This is what makes `git bisect`, `git revert`, and `git cherry-pick` actually useful.
- **Conventional Commits** format (`feat:`, `fix:`, `chore:`, `docs:`, `refactor:`, `BREAKING CHANGE:`) — enables automated changelog generation and semantic-version bumping, and is machine-parseable by CI (Part 04 covers automated release tooling built on this).
- Commit message subject line under ~50 chars, imperative mood (“Add caching layer,” not “Added” or “Adds”), with a blank line and a longer body explaining *why* when the change isn’t self-evident from the diff alone.
- Reference issue/ticket numbers (`Fixes #123`, `JIRA-4567`) for traceability.

ANTI-PATTERNS

- **Giant, unreviewable PRs** spanning dozens of unrelated files — nobody can meaningfully review 2,000 changed lines; split by concern.
- **“WIP” / “fix” / “asdf” commit messages** left in permanent history instead of squashed via interactive rebase before merging.
- **Committing generated/build artifacts** (`dist/`, `node_modules/`, compiled binaries) — bloats the repo and constantly conflicts; use `.gitignore` and build them in CI instead.
- **Force-pushing to shared branches** without `--force-with-lease` and without warning collaborators.
- **Long-lived feature branches** that diverge from `main` for weeks — merge conflicts compound, integration risk grows; prefer small, frequent merges (Section 9.2).

17.2 Code Review Culture

- Review the **diff of the diff** — for a large PR, look at what changed between review rounds, not the whole thing again each time.
- Distinguish **blocking** comments (must fix before merge) from **non-blocking** suggestions (nice-to-have, can be a follow-up) — GitHub’s “Request changes” vs “Comment” review states exist for exactly this.
- A PR author should respond to every comment thread (even if just “done” or explaining why not), and mark threads resolved only once actually addressed — not to silence the noise.

- Aim for review turnaround measured in hours, not days, on active branches — long review latency is one of the biggest hidden costs to team velocity, directly compounding with the long-lived-branch anti-pattern above.

17.3 Repository Hygiene

- A real `README.md`: what the project does, how to run it locally, how to run tests, how to deploy — treat it as the entry point for a new engineer’s first hour.
 - A `CODEOWNERS` file kept current as team structure changes.
 - Branch protection on every branch that matters (Section 14.4), not just `main`.
 - Dead branches pruned regularly (`git branch --merged | grep -v main | xargs git branch -d` as a starting point, reviewed before bulk-deleting).
 - A documented branching strategy (Section 9) that the whole team actually follows — inconsistency here is a constant source of “wait, how do I release this?” confusion.
-

18. Hands-On Labs

LAB ENVIRONMENT

All labs need only Git installed locally and a free GitHub account. Labs 5-7 need a test repository on GitHub (create a new, throwaway public or private repo for this).

Lab 1 — Object Model Exploration

1. Create a new repo, add one file, make one commit.
2. Use `git cat-file -p HEAD`, then follow the tree hash it references with `git cat-file -p <tree-hash>`, then follow the blob hash for your file with `git cat-file -p <blob-hash>` — manually walk the exact object graph Git created.
3. Modify the file slightly and commit again. Compare the new commit's tree hash to the old one — confirm only the changed blob and the trees along the path to it got new hashes; everything else was reused.

Lab 2 — Branching, Merging, and Conflict Resolution

1. Create a repo with a single file containing 10 lines of text.
2. Create two branches from `main`, each editing the same line differently.
3. Merge the first branch into `main` cleanly (fast-forward or otherwise).
4. Merge the second branch into `main` and deliberately produce a conflict; resolve it manually, verify the final file is correct, and complete the merge commit.
5. Redo the same two-branch scenario, but rebase the second branch onto `main` instead of merging — compare the resulting `git log --graph` shapes.

Lab 3 — Recovery Drills

1. Make three commits on a test branch. Run `git reset --hard HEAD~2` (simulate the mistake). Use `git reflog` to recover the two “lost” commits onto a new branch.
2. Stage some changes, then run `git commit --amend` to fold in an additional edit. Confirm via `git log` that the original commit's hash is gone and a new one exists in its place.
3. Deliberately commit a fake “secret” (e.g., `FAKE_API_KEY=abc123` in a file), then practice the full purge workflow from Section 16.4 using `git-filter-repo` on a throwaway local clone — confirm the string is gone from `git log -p --all | grep FAKE_API_KEY` afterward.

Lab 4 — Interactive Rebase Cleanup

1. On a branch, make 5 small commits: two real changes, one “fix typo,” one “oops forgot a file,” and one genuinely unnecessary commit.
2. Use `git rebase -i` to squash/fixup the cleanup commits into their logical predecessors and drop the unnecessary one, ending with 2 clean, atomic commits.
3. Verify the final code state is identical to before the rebase (`git diff` between the branch tip before and after should be empty), while `git log --oneline` is now much cleaner.

Lab 5 — GitHub Collaboration Workflow

1. Create a repo on GitHub, clone it, set up branch protection on `main` requiring at least one PR review.
2. Create a feature branch, push it, open a PR using `gh pr create`.
3. Add a `CODEOWNERS` file that requires review on a specific path; make a change to that path in a new PR and confirm the correct reviewer is automatically requested.
4. Practice all three merge strategies (Section 4.4) on three separate small PRs, and compare the resulting `main` history for each.

Lab 6 — Git Hooks and pre-commit

1. Set up the `pre-commit` framework (Section 12.3) with `detect-private-key` and `trailing-whitespace` hooks in a test repo.
2. Attempt to commit a file containing a fake private key block (`-----BEGIN RSA PRIVATE KEY-----`) and confirm the commit is blocked locally.
3. Bypass it with `git commit --no-verify` and observe that it succeeds — demonstrating why local hooks alone aren’t sufficient enforcement (tie this back to Section 15.3’s discussion of GitHub’s server-side push protection).

Lab 7 — Bisect a Real Regression

1. Create a small script-based “project” with ~15 commits, where one commit in the middle introduces a deliberate bug (e.g., a function starts returning the wrong value) and a simple test script that can detect it.
 2. Use `git bisect start`, mark the current commit bad and the first commit good, and manually walk through `git bisect good / bad` to find the exact culprit.
 3. Redo the same search using `git bisect run ./test_script.sh` and confirm it lands on the identical commit automatically.
-

19. Command & Reference Cheat Sheet

19.1 Everyday Commands

Command	Purpose
<code>git status</code>	Current state across all three areas
<code>git add -p</code>	Stage selectively, by hunk
<code>git commit -m "msg"</code>	Commit staged changes
<code>git log --oneline --graph --all</code>	Visual history across all branches
<code>git diff</code> / <code>git diff --staged</code>	Unstaged / staged changes
<code>git switch -c <branch></code>	Create + switch to a new branch
<code>git push -u origin <branch></code>	Push + set tracking (first push of a branch)

19.2 Undo & Recovery

Command	Purpose
<code>git restore <file></code>	Discard uncommitted changes to a file
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes staged
<code>git revert <hash></code>	Safely undo a commit on a SHARED branch
<code>git reflog</code>	Recovery safety net — find “lost” commits
<code>git stash</code> / <code>git stash pop</code>	Temporarily shelve / restore uncommitted work
<code>git cherry-pick <hash></code>	Apply one specific commit elsewhere

19.3 Rebase & History

Command	Purpose
<code>git rebase -i HEAD~N</code>	Interactively edit the last N commits
<code>git rebase main</code>	Replay current branch on top of main
<code>git rebase --abort</code>	Bail out of a rebase, no harm done
<code>git push --force-with-lease</code>	Safe force-push after history rewrite
<code>git bisect start</code> / <code>good</code> / <code>bad</code>	Binary-search for a regression

19.4 GitHub CLI

Command	Purpose
<code>gh pr create</code>	Open a PR from the current branch
<code>gh pr checkout <n></code>	Check out a PR locally, even from a fork
<code>gh pr merge <n> --squash --delete-branch</code>	Merge + clean up
<code>gh run watch</code>	Live-follow the current Actions run
<code>gh api <endpoint></code>	Raw GitHub API access

20. Production Readiness Checklist

- ☐ `main` (and any release branches) has branch protection: required PR + approvals, required status checks, up-to-date-before-merge, no force-push/direct-push
- ☐ `CODEOWNERS` file exists and reflects current team ownership
- ☐ Secret scanning + push protection enabled on the repository
- ☐ Dependabot alerts (and ideally security updates) enabled
- ☐ A documented, team-wide branching strategy exists and is actually followed (Section 9)
- ☐ Commit signing required on protected branches, or at minimum encouraged and configured for all engineers
- ☐ CI-facing automation uses scoped tokens (GitHub App / `GITHUB_TOKEN`), not a personal PAT belonging to one engineer
- ☐ `.gitignore` covers all build artifacts, dependencies, and local environment files; nothing generated is committed
- ☐ Large binary assets, if any, are in Git LFS or an external artifact store — not committed raw
- ☐ `pre-commit` (or equivalent) hooks are version-controlled and documented for local setup, backed by the same checks enforced in CI
- ☐ A known, documented incident procedure exists for a leaked secret (rotate first, always) — not something the team improvises mid-incident
- ☐ Stale branches are pruned on a regular cadence; repository size/history is periodically reviewed